Documentation

Team C4
07/07/2026

1 Team members

Moeez Mufti
Muhammad Hamas
Ali Sohail
Talha Khan

2 Introduction

This project applies IoT and embedded systems concepts to automated food quality control. It combines machine vision, distributed embedded controllers, sensors, actuators, and wireless communication to image a food item from multiple angles and classify it as pass or reject.

The target application is a "QC station" prototype for automated food inspection. For this prototype, an operator places a food item on a motorized turntable and presses a start button. The item is rotated to four fixed angles and photographed at each one, producing a consistent multi-view image set for that inspection. Each image set is stored per inspection and evaluated by an edge-AI classifier (a TensorFlow Lite model running locally on the master device) that returns a pass/reject judgement for every view. An item is rejected if mold is detected from any angle, and passed only if every angle is clean.

The system simulates an end-of-line inspection step in food packaging, replacing manual visual mold checking, which is slow and inconsistent between inspectors and prone to missing mold on faces not immediately visible, with a repeatable, automated capture-and-classify cycle.

3 Concept description

The target application is an automatic quality-control inspection station. The prototype is designed to inspect a physical part by rotating it to four fixed viewing angles, capturing one image at each angle, and automatically classifying the captured images as either **PASS** or **REJECT** using a TensorFlow Lite model.

The system is coordinated by a **Raspberry Pi**, which runs the Flask application, the web dashboard, the MQTT client logic, the image storage logic, and the automatic classifier. The Raspberry Pi also communicates with a local Mosquitto MQTT broker. The Arduino Uno WiFi Rev2 controls the turntable servo motor, while the ESP32-CAM captures and uploads the inspection images.

An inspection can be started in two ways. First, the operator can press a physical pushbutton connected to the Arduino on pin 2. The Arduino then sends an HTTP request to the Raspberry Pi. Second, the user can start the inspection from the Flask web dashboard. In both cases, the Raspberry Pi receives a request at:

POST /api/inspection/start

After the inspection begins, the Raspberry Pi sends HTTP commands to the Arduino to move the turntable. The part is rotated to four fixed angles:

0°, 60°, 120°, 180°

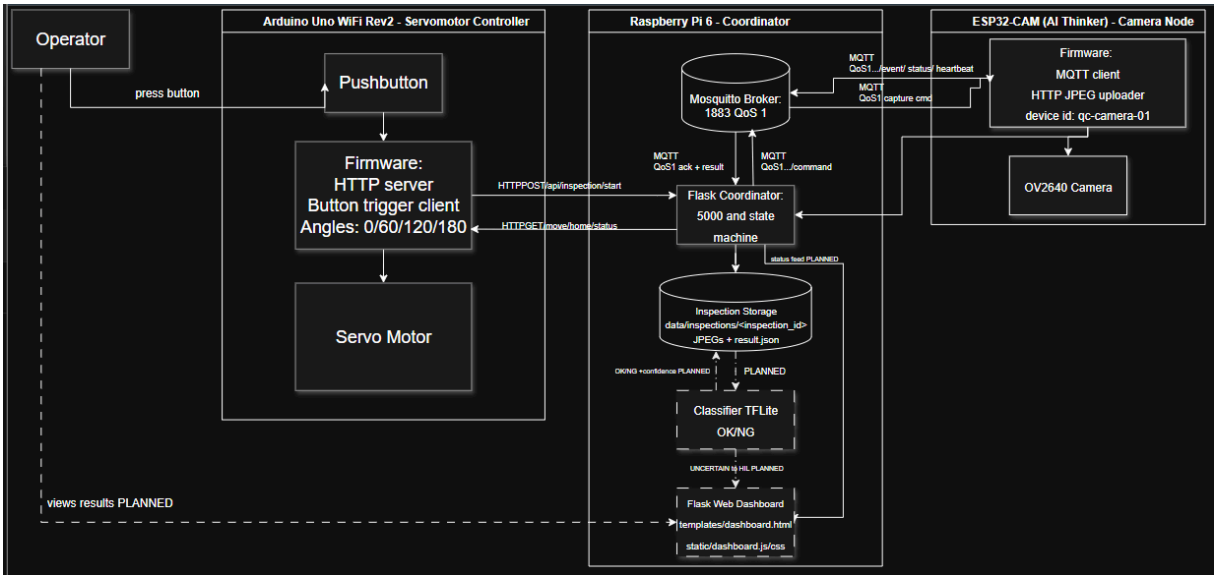
At each angle, the Raspberry Pi sends an MQTT capture command to the ESP32-CAM. The ESP32-CAM captures a JPEG image using the OV2640 camera sensor and uploads the raw image data back to the Raspberry Pi through HTTP. Once each image is saved, the Flask application runs the saved JPEG through the TensorFlow Lite classifier. The model returns a predicted label, either **PASS** or **REJECT**, together with a confidence score.

The final inspection result is then calculated automatically:

All four views PASS → final result PASS
At least one view REJECT → final result REJECT
Hardware, communication, capture, or validation failure → SYSTEM_ERROR

Each inspection is stored in its own directory inside **`data/inspections/`**. The folder contains the four captured JPEG images and a **`result.json`** file containing the inspection ID, state, image filenames, classification results, confidence scores, timestamps, final result, and possible error information.

Block Diagram of the Target Application



The block diagram should show the **Raspberry Pi** as the central coordinator. The Raspberry Pi runs the Flask server, the web dashboard, the MQTT client logic, the automatic defect classifier, and the inspection storage system. It communicates with the Arduino over plain HTTP and with the ESP32-CAM through MQTT and HTTP.

The Arduino Uno WiFi Rev2 is responsible for the mechanical turntable. It reads the physical pushbutton and controls the servo motor. The ESP32-CAM is responsible for image capture.

It receives MQTT capture commands, captures JPEG images, and uploads them back to the Raspberry Pi.

Main Application of the Prototype

The main application of this prototype is to automate a small visual inspection process. In a manual inspection setup, a technician would need to rotate the part, take pictures, compare the images, and decide whether the part passes or fails. This prototype automates most of that workflow.

The system automatically:

- starts an inspection,
- rotates the part to fixed angles,
- captures four images,
- stores the images,
- classifies each image using a TFLite model,
- displays the result on a Flask dashboard,
- and saves the final inspection record.

This makes the prototype suitable for demonstrating embedded system coordination, actuator control, camera-based sensing, MQTT communication, HTTP APIs, Flask-based monitoring, and edge AI classification on a Raspberry Pi.

4 Project/Team management

Team Member	*Tasks*
Muhammad Hamas	*System Architecture, Inference, Data Collection, Dashboard*
Muhammad Ali	*Programming and Debugging, Dashboard*
Moeez Mufti	*Schematics, Structure & Wiring*
Talha Khan	*Communication & Data Collection/Verification/Cleaning*

5 Technologies

Technological Approaches

Sensors and actuators

- **Push button**: detects the operator's start signal via a simple GPIO pin, debounced in software.

- **Positional servo motor**: rotates the turntable to four fixed angles (0°, 60°, 120°, 180°). Controlled by the Arduino over PWM using the Servo library.
- **ESP32-CAM image sensor**: an OV2640-based camera module wired directly to the ESP32-CAM's parallel camera interface. Captures JPEG frames on command.

Communication protocols

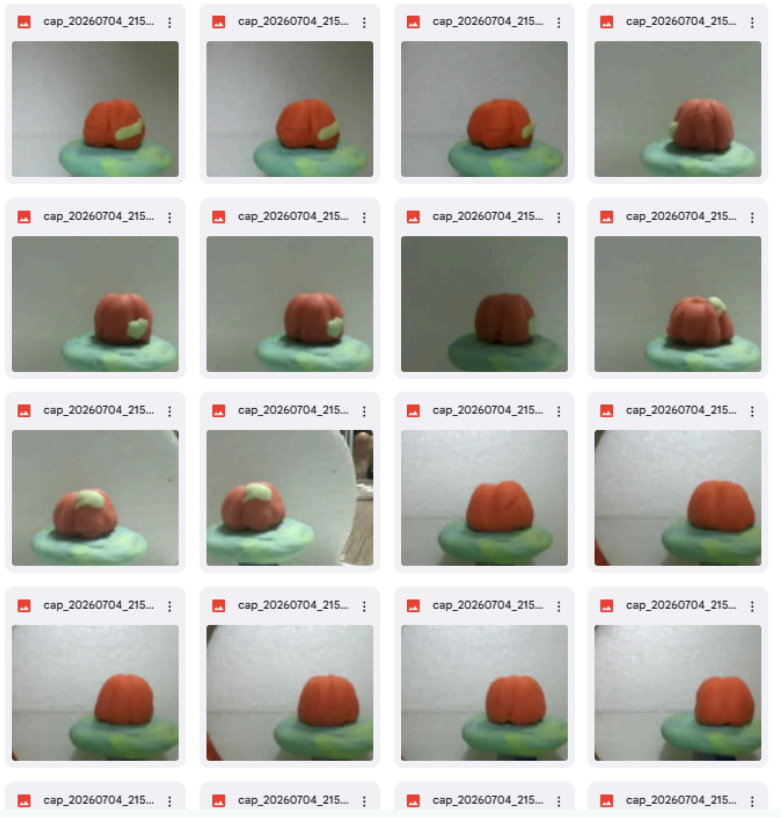
- **Wi-Fi (802.11)**: both the Arduino Uno WiFi Rev2 and the ESP32-CAM connect to a local wireless network to reach the Raspberry Pi.
- **HTTP**: used for request/response interactions where an immediate answer is expected: the Arduino exposes `/move` and `/home` routes for the Pi to control the turntable, and the ESP32-CAM uploads captured JPEG images to the Pi as raw HTTP POST bodies.
- **MQTT**: used for the capture command/event exchange between the Pi and the ESP32-CAM. The Pi publishes a capture command to a command topic; the camera publishes acknowledgement and result events (accepted, capture started, capture complete or failed) back on an event topic. A local Mosquitto broker running on the Raspberry Pi handles all MQTT traffic.
- **JSON**: the message format for all MQTT payloads and HTTP API bodies, keeping every exchange structured and easy to parse on both the microcontroller and server side.

Programming languages and frameworks

- **C++ (Arduino/ESP32 framework)**: firmware for both the Arduino Uno WiFi Rev2 (turntable and button control, using NINA and Servo) and the ESP32-CAM (camera capture and MQTT messaging, using ArduinoMqttClient and ArduinoJson).
- **Python**: the Raspberry Pi server, built with Flask for the HTTP API and dashboard, and paho-mqtt for the MQTT client side of the camera communication.
- **JavaScript, HTML, CSS**: the browser-based dashboard, which polls the Pi's status API and renders live inspection state and images.

Edge AI / inference

TensorFlow Lite (TFLite): the mold-detection model runs directly on the Raspberry Pi using a lightweight TFLite interpreter, so classification happens on-device with no cloud dependency or network round trip. The model takes a single image as input and returns a pass/reject classification with a confidence score.



Some examples of images from the dataset, collected with different angles and lighting.

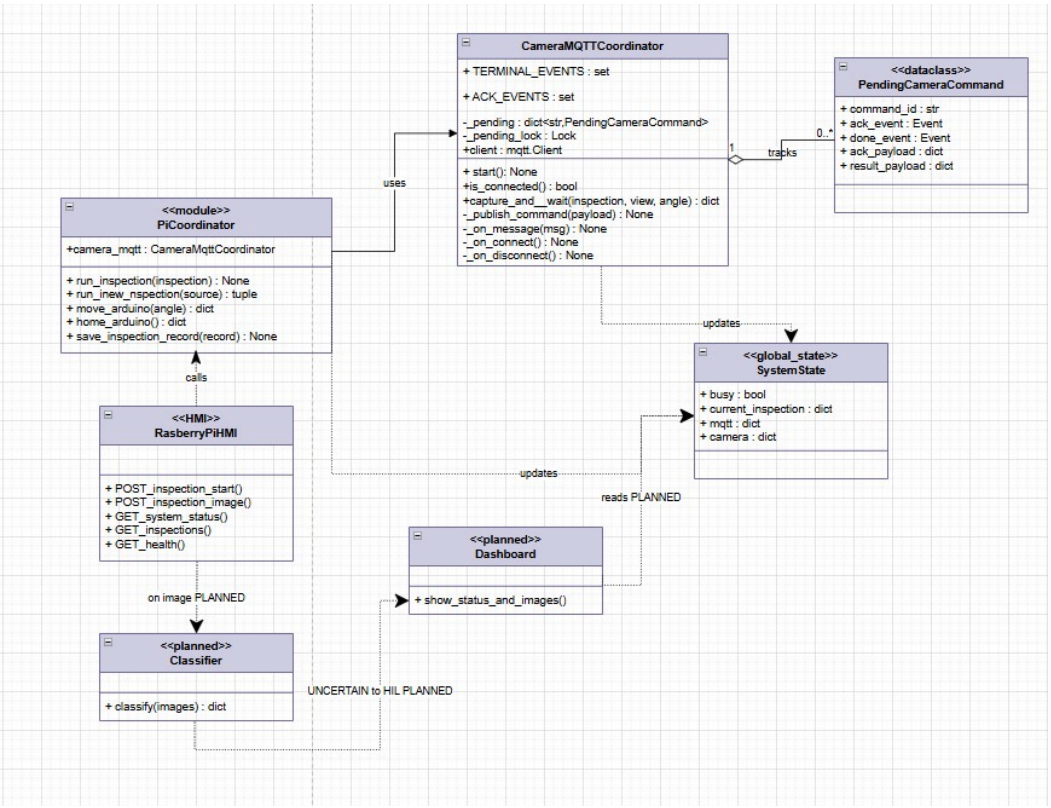
6 Implementation

Static Structure of the Environment

The environment is composed of three physical devices and a layered software structure that runs primarily on the Raspberry Pi.

Physical components

- **Arduino Uno WiFi Rev2**: hosts the operator push button and the turntable's positional servo. Runs a small HTTP server exposing `/move` and `/home`, and calls out to the Pi when the button is pressed.
- **ESP32-CAM**: hosts the camera sensor. Connects to the Pi's MQTT broker to receive capture commands and publish status/event messages, and uploads captured JPEGs to the Pi over HTTP.
- **Raspberry Pi**: the central controller. Runs the Mosquitto MQTT broker, the Flask application, and the TFLite inference engine. Coordinates the Arduino and ESP32-CAM and serves the operator dashboard.



Software structure

The Pi-side application is organized into a thin routing layer and a set of independent service modules, each responsible for one external concern:

Module	**Responsibility**
`app.py`	Flask routes, inspection state machine, and the capture-classify sequence that ties the other modules together
`services/arduino.py`	HTTP client for the turntable (`/move`, `/home`)
`services/camera_mqtt.py`	MQTT client managing capture commands and their acknowledgement/result events
`services/classifier.py`	Loads the TFLite model and performs preprocessing and inference on each captured image
`services/storage.py`	Persists each inspection as a folder containing its images and a `result.json` record
`templates/`, `static/`	Dashboard page, styling, and client-side polling/rendering logic

`app.py` holds a single in-memory inspection record at a time, guarded by a lock, since the station processes one item per inspection cycle. Each inspection record carries its state

(`CAPTURING`, `COMPLETE`, `ERROR`), the list of per-view results (image filename, model label, confidence, and score breakdown), and the final PASS/REJECT/SYSTEM_ERROR outcome. This record is written to disk after every update, so the on-disk `result.json` always mirrors what the dashboard displays.

This separation means each service module can be modified or replaced (for example, swapping the classifier for a different model format, or the camera link for a different protocol) without changes to the other modules.

6.2 Use Cases: Automated Inspection Workflow

Actors

- **Operator:** Initiates the inspection (via physical button or web dashboard) and monitors progress.
- **System:** The automated Pi-Arduino-ESP32-CAM chain that executes the inspection cycle without further manual intervention.

Process Description

The inspection process is a fully automated cycle designed to minimize manual labor and increase consistency. The workflow proceeds as follows:

1. **Initiation:** The operator places a food item on the turntable and triggers the start command. The system verifies that it is in an idle state and that all hardware components (Raspberry Pi, Arduino, ESP32-CAM) are reachable.
2. **Execution:** The system coordinates the inspection by signaling the Arduino to rotate the turntable to four fixed angles (0°, 60°, 120°, 180°). At each designated position, the system commands the ESP32-CAM to capture and upload a high-resolution image to the Raspberry Pi.
3. **Analysis:** As each image is received, the system utilizes the local TensorFlow Lite model to perform real-time classification. Each view is processed and recorded with a "PASS" or "REJECT" label, alongside a corresponding confidence score.
4. **Determination of Final Result:** Upon the completion of all four image captures and classifications, the system aggregates the findings. The final outcome is determined as "REJECT" if any single view shows signs of mold, and "PASS" only if every view is clean.
5. **Monitoring:** Throughout the sequence, the operator can view the real-time status on the dashboard, including the current inspection state, live captured images, per-view classification confidence, and the final result.

7 Results

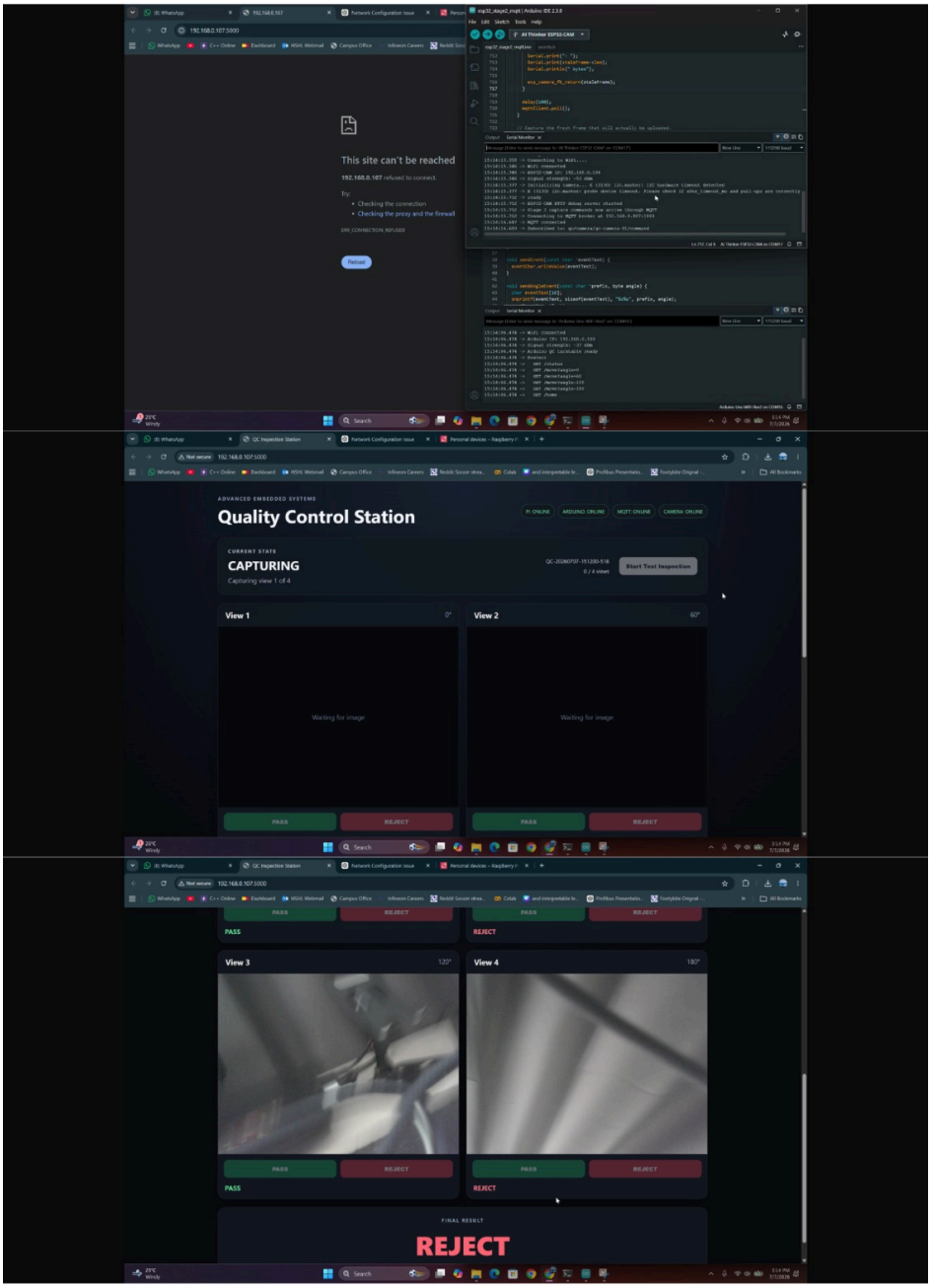
The dashboard test shows that the QC Station was able to run an inspection cycle and display the system status through the Flask web interface. At the beginning, the browser could not connect to the Raspberry Pi server, which indicated that the Flask application was not yet reachable at the configured IP address and port. After the server and device connections were running, the dashboard successfully loaded at our home IP address.

The dashboard shows that all main four devices are online.This confirms that the Raspberry Pi Flask server, Arduino turntable controller, Mosquitto MQTT broker, and ESP32-CAM were communicating with the system.

During the inspection, the station entered the **CAPTURING** state and began capturing the four required views. The dashboard displayed the inspection ID, progress counter, and individual image cards for the four turntable angles: 0, 60, 120, 180. The screenshots show that the inspection images were received and displayed on the dashboard. After the views were reviewed/classified, the system produced the final inspection result: REJECT.

This result means that at least one of the inspected views was classified or marked as defective. Therefore, according to the system rule, the complete inspection is rejected.

Overall, the result demonstrates that the prototype successfully achieved its main workflow: device connection, inspection start, multi-view image capture, dashboard display, per-view judgement, and final PASS/REJECT decision.



8 Fail-Safe Manual Inspection Option

In addition to the automatic inspection version, a second implementation was developed as a **fail-safe manual inspection option**. This version is included in the `QC Station Manual Inspection` ZIP file. It uses the same physical inspection station, the same Raspberry Pi coordinator, the same Arduino turntable control, and the same ESP32-CAM image capture workflow. However, it removes the TensorFlow Lite classification step and replaces it with manual technician review through the Flask dashboard.

The purpose of this version is to provide a reliable backup method if the automatic classifier cannot be used. The automatic version depends on the trained model, image quality, lighting

conditions, camera focus, and correct preprocessing. If the model is not accurate enough, unavailable, or unsuitable for a certain test case, the inspection station should still be usable. The manual version keeps the core inspection process working and allows a human operator to make the final PASS or REJECT decision.

The manual inspection system is still based on the same distributed embedded architecture. The Raspberry Pi remains the central controller and runs the Flask application, the dashboard, the MQTT communication logic, and the inspection storage system. The Arduino Uno WiFi Rev2 controls the turntable servo motor and detects the physical start button. The ESP32-CAM captures the inspection images and uploads them to the Raspberry Pi.

An inspection can be started either by pressing the physical button connected to the Arduino or by pressing the Start button on the Flask dashboard. In both cases, the Raspberry Pi receives the request through `POST /api/inspection/start`. After the inspection starts, the Raspberry Pi sends HTTP commands to the Arduino to rotate the turntable to four fixed angles: `0°`, `60°`, `120°`, and `180°`

At each angle, the Raspberry Pi sends an MQTT capture command to the ESP32-CAM. The camera captures a JPEG image and uploads it back to the Raspberry Pi using HTTP. The Flask server validates the upload by checking the view index, servo angle, command ID, content type, and JPEG data. After validation, the image is stored in the inspection folder.

Where the manual method differs from the automatic method is after the image capture stage. In the automatic version, each saved image is immediately passed into the TensorFlow Lite classifier, which produces a predicted label and confidence score. In the manual fail-safe version, no classifier is used. Instead, after all four images have been captured, the system enters `WAITING_FOR_REVIEW` and displays the images on the dashboard.

The technician then reviews each captured image manually. For every view, the dashboard provides PASS and REJECT buttons. When the technician selects a result, the dashboard sends the decision to `POST /api/inspection/<id>/classify/<view>` with the selected label. After at least one image has been reviewed but not all four are complete, the inspection enters `REVIEW_IN_PROGRESS`.

Once all four views have been reviewed, the Raspberry Pi calculates the final result. If all four images are marked PASS, the final result is PASS. If any image is marked REJECT, the final result is REJECT. If a hardware, communication, capture, or upload failure occurs, the result is set to `SYSTEM_ERROR`.

The final inspection record is stored in `data/inspections/`. Each inspection folder contains the four JPEG images and a `result.json` file. This file records the inspection ID, captured views, manual classifications, timestamps, final result, and any error information.

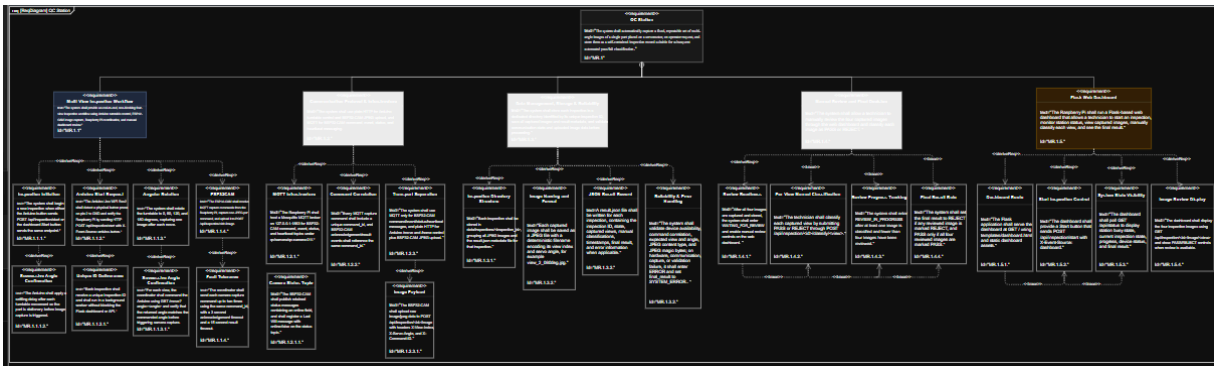
Overall, this manual inspection version provides a practical fail-safe option for the project. It keeps the same hardware setup and communication workflow as the automatic system, but moves the final decision from the AI model to the technician. This ensures that the prototype remains usable even when automatic classification is not reliable or not available.

9 Sources/References

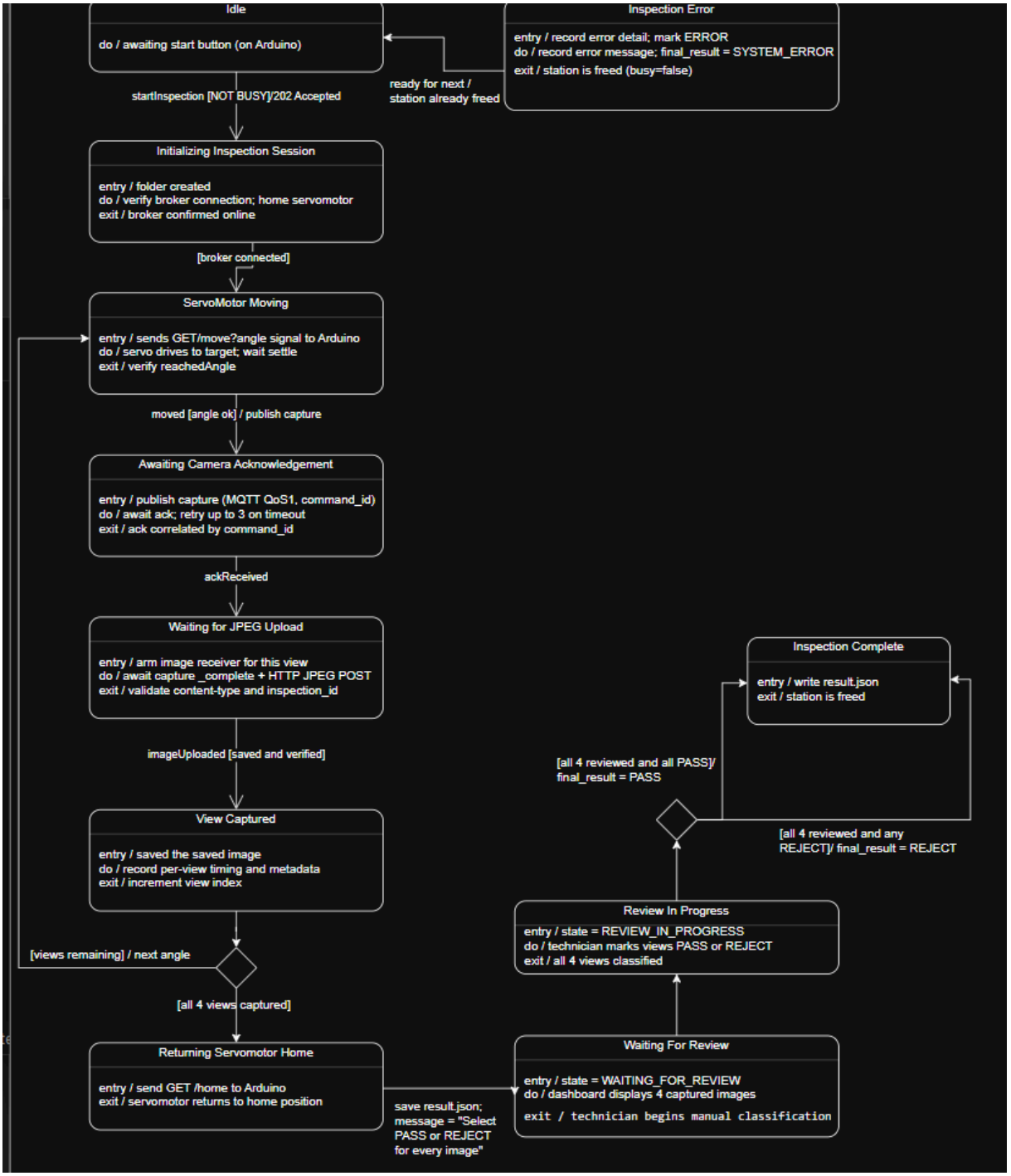
Repository
https://github.com/MAliSohail/Team-C4-Advanced-Embedded-Systems-Lab/tree/main
Training Dataset:
https://drive.google.com/drive/folders/17IENgQY0aweURkh9UtZ6BOD2HO2dTbPg?usp=sharing

Appendix

Requirements Diagram



State Machine Diagram


Idle
do / awaiting start button (on Arduino)
startInspection [NOT BUSY]/202 Accepted
Inspection Error
entry / record error detail; mark ERROR
do / record error message; final_result = SYSTEM_ERROR
exit / station is freed (busy=false)
ready for next /
station already freed
Initializing Inspection Session
entry / folder created
do / verify broker connection; home servomotor
exit / broker confirmed online
[broker connected]
ServoMotor Moving
entry / sends GET/move?angle signal to Arduino
do / servo drives to target; wait settle
exit / verify reachedAngle
moved [angle ok] / publish capture
Awaiting Camera Acknowledgement
entry / publish capture (MQTT QoS1, command_id)
do / await ack; retry up to 3 on timeout
exit / ack correlated by command_id
ackReceived
Waiting for JPEG Upload
entry / arm image receiver for this view
do / await capture _complete + HTTP JPEG POST
exit / validate content-type and inspection_id
imageUploaded [saved and verified]
View Captured
entry / saved the saved image
do / record per-view timing and metadata
exit / increment view index
[views remaining] / next angle
[all 4 views captured]
Returning Servomotor Home
entry / send GET /home to Arduino
exit / servomotor returns to home position
save result.json;
message = "Select
PASS or REJECT
for every image"
Waiting For Review
entry / state = WAITING_FOR_REVIEW
do / dashboard displays 4 captured images
exit / technician begins manual classification
Review In Progress
entry / state = REVIEW_IN_PROGRESS
do / technician marks views PASS or REJECT
exit / all 4 views classified
[all 4 reviewed and all PASS]/
final_result = PASS
[all 4 reviewed and any
REJECT]/ final_result = REJECT
Inspection Complete
entry / write result.json
exit / station is freed